SQL & Python Business Analytics Project

SQL & Python Data Analytics Project

Insights & Visual Analysis

Project Objective

- Solve 15 business problems
- Execute queries in SQL
- Solve same problems in Python
- Validate that outputs are identical
- Demonstrate technical accuracy & analytical consistency

BASIC PROBLEMS

1) List all unique cities where customers are located

- SQL Code:

```sql
#1. List all unique cities where customers are located
SELECT DISTINCT customer_city
FROM customers
ORDER BY customer_city;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

customer_city
abadia dos dourados
abadiania
abaete
abaetetuba
abaiara
abaira
abare
abatia
abdon batista
abelardo luz
abrantes

customers 1

- Python Code:

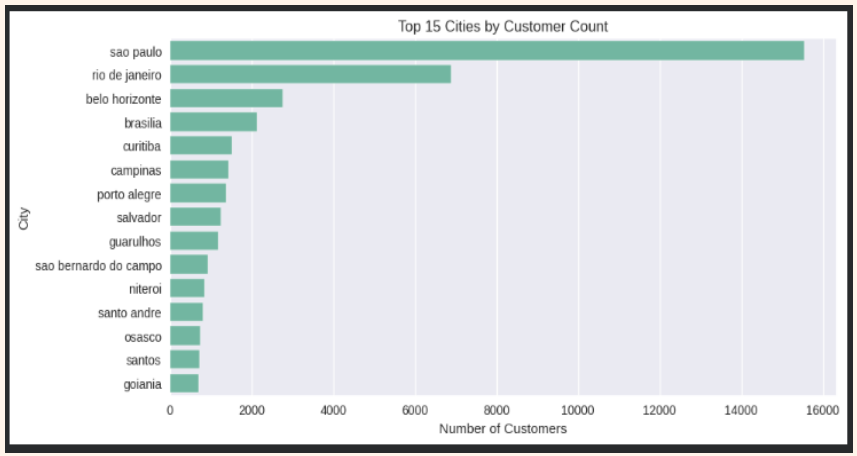
1) List all unique cities where customers are located

```python
unique_cities = customers['customer_city'].dropna().unique()
sorted(unique_cities)
```

```
'charqueadas',
'charrua',
'chaval',
'chavantes',
'chaveslandia',
'chiapetta',
'chopinzinho',
'chorozinho',
'chorrocho',
'chui',
'cianorte',
'cicero dantas',
'cidade gaucha',
'cidade ocidental',
'cidelandia',
'cidreira',
'cipo',
'cipo-guacu',
'ciriaco',
'claraval',
'claro dos pocoes',
'claudia',
'claudio',
'clementina',
'clevelandia',
'coaraci',
'coari',
'cocais',
```

```python
len(unique_cities)
```

```
4119
```



2) Count the number of orders placed in 2017

- SQL Code:

```sql
#2. Count the number of orders placed in 2017
SELECT COUNT(*) AS total_orders_2017
FROM orders
WHERE order_purchase_timestamp >= '2017-01-01'
  AND order_purchase_timestamp < '2018-01-01';
```

Result Grid Filter Rows:

	total_orders_2017
▶	45101

- Python Code:

2) Count number of orders placed in 2017

```python
orders['order_purchase_timestamp'] = pd.to_datetime(orders['order_purchase_timestamp'])

orders_2017 = orders[orders['order_purchase_timestamp'].dt.year == 2017]
len(orders_2017)
```

```
45101
```

3) Find the total sales per category

- SQL Code:

```sql
#3. Find the total sales per category
SELECT
    p.product_category AS category,
    SUM(oi.price) AS total_sales
FROM order_items oi
JOIN products p
    ON oi.product_id = p.product_id
GROUP BY p.product_category
ORDER BY total_sales DESC;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

category	total_sales
HEALTH BEAUTY	1258681.3399999724
Watches present	1205005.6799999962
bed table bath	1036988.6800000721
sport leisure	988048.9700000428
computer accessories	911954.3200000391
Furniture Decoration	729762.4900000372
Cool Stuff	635290.8500000002
housewares	632248.6600000234
automotive	592720.1100000131
Garden tools	485256.46000001667
toys	483946.6000000019
babies	411764.89000000525
perfumery	399124.869999999

Result 3

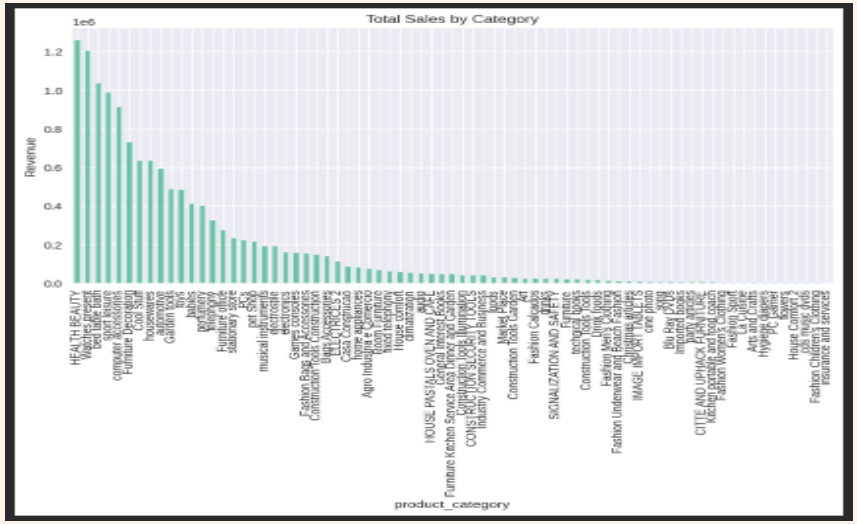
- Python Code:

3) Total sales per category

```python
sales_per_category = (
    merged.groupby('product_category')['price']
    .sum()
    .reset_index(name='total_sales')
    .sort_values(by='total_sales', ascending=False)
)

sales_per_category
```

	product_category	total_sales
30	HEALTH BEAUTY	1258681.34
45	Watches present	1205005.68
49	bed table bath	1036988.68
68	sport leisure	988048.97
53	computer accessories	911954.32
---	---	---
58	flowers	1110.04
32	House Comfort 2	760.27
50	cds music dvds	730.00
18	Fashion Children's Clothing	569.85
62	insurance and services	283.29

73 rows × 2 columns



4) Calculate the percentage of orders paid in installments

- SQL Code:

```sql
#4. Calculate the percentage of orders paid in installments
SELECT
    COUNT(DISTINCT CASE
        WHEN payment_installments > 1 THEN order_id
    END) * 100.0
    / COUNT(DISTINCT order_id)
    AS installment_percentage
FROM payments;
```

Result Grid | Filter Rows:

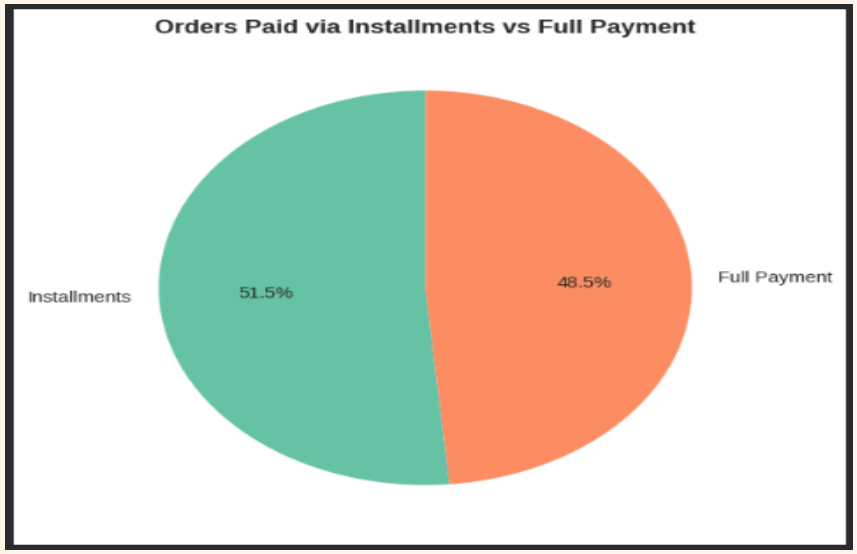
	installment_percentage
▶	51.45817

- Python Code:

4) Percentage of orders paid in installments

```python
installment_orders = payments.loc[
    payments['payment_installments'] > 1,
    'order_id'
].nunique()

total_orders = payments['order_id'].nunique()

installment_percentage = (installment_orders / total_orders) * 100

installment_percentage
```

```
51.45816572807723
```



5) Count the number of customers from each state

- SQL Code:

```sql
#5. Count the number of customers from each state
SELECT
    customer_state,
    COUNT(DISTINCT customer_unique_id) AS total_customers
FROM customers
GROUP BY customer_state
ORDER BY total_customers DESC;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

customer_state	total_customers
SP	40302
RJ	12384
MG	11259
RS	5277
PR	4882
SC	3534
BA	3277
DF	2075
ES	1964
GO	1952
PE	1609
CE	1313

Result 5

- Python Code:

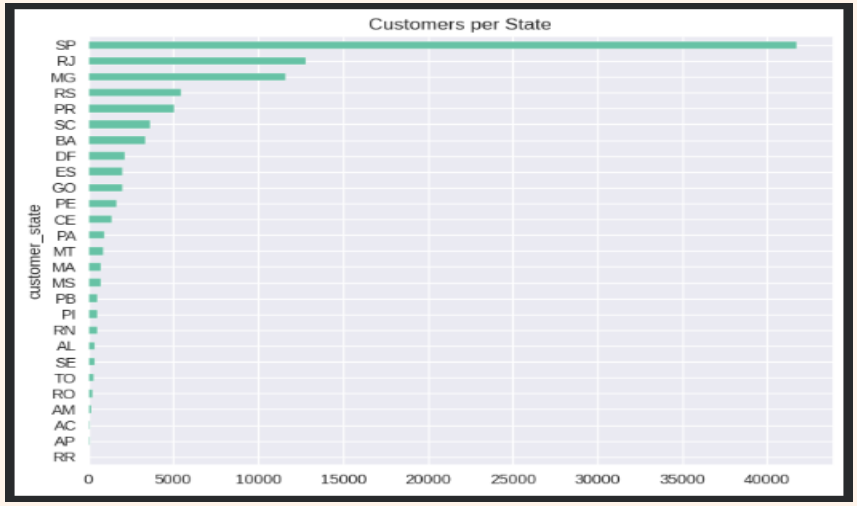
5) Number of customers from each state

```python
customers_by_state = (
    customers.groupby('customer_state')
    .size()
    .sort_values(ascending=False)
)

customers_by_state
```

customer_state	0
SP	41746
RJ	12852
MG	11635
RS	5466
PR	5045
SC	3637
BA	3380
DF	2140
ES	2033
GO	2020
PE	1652

```python
len(customers_by_state)
```

27



INTERMEDIATE PROBLEMS

1) Calculate the number of orders per month in 2018

- SQL Code:

```sql
#1. Calculate the number of orders per month in 2018
SELECT
    MONTH(order_purchase_timestamp) AS month,
    COUNT(order_id) AS total_orders
FROM orders
WHERE YEAR(order_purchase_timestamp) = 2018
GROUP BY MONTH(order_purchase_timestamp)
ORDER BY month;
```

Result Grid | Filter Rows:

month	total_orders
1	7269
2	6728
3	7211
4	6939
5	6873
6	6167
7	6292
8	6512
9	16
10	4

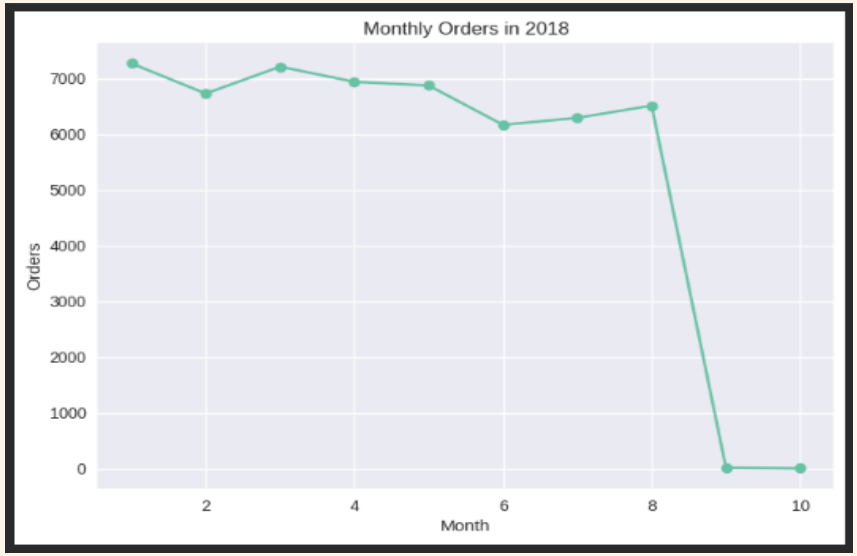
- Python Code:

1) Orders per month in 2018

```python
orders_2018 = orders[orders['order_purchase_timestamp'].dt.year == 2018]
orders_2018['month'] = orders_2018['order_purchase_timestamp'].dt.month
orders_2018.groupby('month').size().sort_index()
```

month	0
1	7269
2	6728
3	7211
4	6939
5	6873
6	6167
7	6292
8	6512
9	16
10	4

dtype: int64



2) Find the average number of products per order, grouped by customer city

- SQL Code:

```sql
#2. Find the average number of products per order, grouped by customer city
SELECT
    customer_city,
    AVG(order_product_count) AS avg_products_per_order
FROM (
    SELECT
        o.order_id,
        customer_city,
        COUNT(oi.product_id) AS order_product_count
    FROM orders o
    JOIN customers c
        ON o.customer_id = c.customer_id
    JOIN order_items oi
        ON o.order_id = oi.order_id
    GROUP BY o.order_id, customer_city
) sub
GROUP BY customer_city
ORDER BY avg_products_per_order DESC;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

customer_city	avg_products_per_order
padre carvalho	7.0000
celso ramos	6.5000
datas	6.0000
candido godoi	6.0000
matias olimpio	5.0000
cidelandia	4.0000
picarra	4.0000
curralinho	4.0000
teixeira soares	4.0000
morro de sao paulo	4.0000

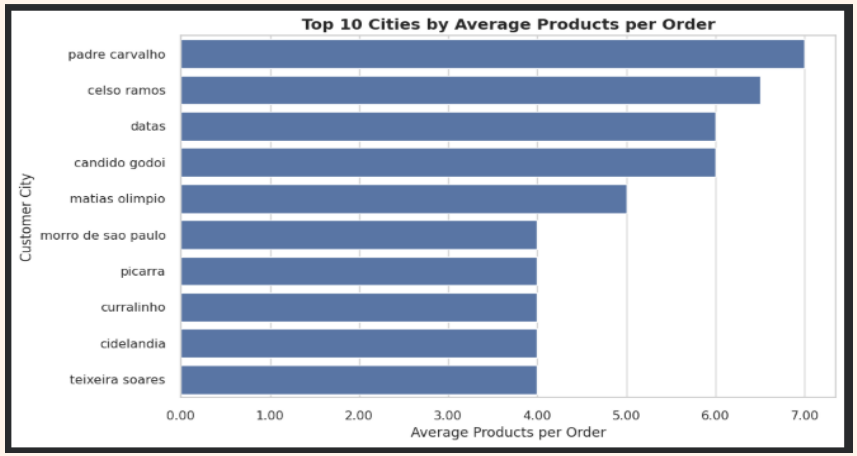
Result 7

- Python Code:

```python
merged = orders.merge(customers, on='customer_id') \
               .merge(order_items, on='order_id')

order_product_count = (
    merged.groupby(['order_id', 'customer_city'])['product_id']
    .count()
    .reset_index(name='order_product_count')
)

avg_products_per_city = (
    order_product_count.groupby('customer_city')['order_product_count']
    .mean()
    .reset_index(name='avg_products_per_order')
    .sort_values(by='avg_products_per_order', ascending=False)
)
avg_products_per_city
```

	customer_city	avg_products_per_order
2619	padre carvalho	7.0
907	celso ramos	6.5
1154	datas	6.0
756	candido godoi	6.0
2264	matias olimpio	5.0
...	...	...
1946	jatauba	1.0
1947	jatoba	1.0
1949	jaua	1.0
1950	jaupaci	1.0
1935	jardim	1.0

4110 rows × 2 columns



3) Calculate the percentage of total revenue contributed by each product category

- SQL Code:

```sql
#3. Calculate the percentage of total revenue contributed by each product category
SELECT
    p.product_category AS category,
    ROUND(
        SUM(oi.price) * 100.0 /
        (SELECT SUM(price) FROM order_items),
        2
    ) AS revenue_percentage
FROM order_items oi
JOIN products p
    ON oi.product_id = p.product_id
GROUP BY p.product_category
ORDER BY revenue_percentage DESC;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

category	revenue_percentage
HEALTH BEAUTY	9.26
Watches present	8.87
bed table bath	7.63
sport leisure	7.27
computer accessories	6.71
Furniture Decoration	5.37
Cool Stuff	4.67
housewares	4.65
automotive	4.36
Garden tools	3.57

Result 8

- Python Code:

3) % revenue contribution by each category

```python
base = (
    order_items[['order_id','product_id','price']]
    .merge(products[['product_id','product_category']],
           on='product_id',
           how='inner')
)

category_sales = base.groupby('product_category')['price'].sum()

total_sales = order_items['price'].sum()  # exact SQL equivalent

percentage = (
    category_sales * 100.0 / total_sales
).round(2).sort_values(ascending=False)

percentage
```

product_category	price
HEALTH BEAUTY	9.26
Watches present	8.87
bed table bath	7.63
sport leisure	7.27
computer accessories	6.71
...	...
Hygiene diapers	0.01
flowers	0.01
PC Gamer	0.01
Fashion Children's Clothing	0.00
Insurance and services	0.00

73 rows × 1 columns

dtype: float64

4) Identify the correlation between product price and number of times purchased

- SQL Code:

```sql
#4. Identify the correlation between product price and number of times purchased
SELECT
    oi.product_id,
    AVG(oi.price) AS avg_price,
    COUNT(oi.order_id) AS purchase_count
FROM order_items oi
GROUP BY oi.product_id;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: | Fetch

product_id	avg_price	purchase_count
4244733e06e7ecb4970a6e2683c13e61	59.23333333333332	9
e5f2d52b802189ee658865ca93d83a8f	239.9	1
c777355d18b72b67abbeef9df44fd0fd	199	3
7634da152a4610f1595efa32f14722fc	12.99	2
ac6c3623068f30de03045865e4e10089	202.40000000000006	12
ef92defde845ab8450f9d70c526ef70f	21.9	5
8d4f2bb7e93e6710a28f34fa83ee7d28	18.566666666666666	3
557d850972a7d6f792fd18ae1400d9b6	810	1
310ae3c140ff94b03219ad0adc3c778f	145.95	2
4535b0e1091c278dfd193e5a1d63b39f	53.99	4

Result 9

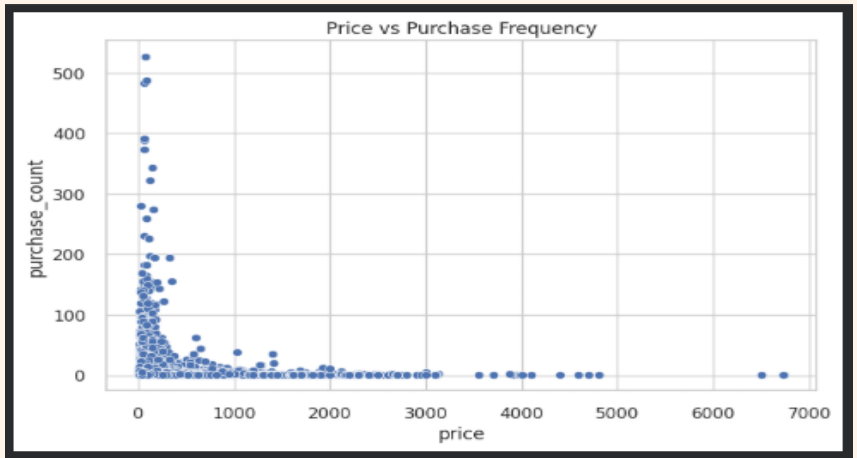
- Python Code:

4) Correlation between price and purchase frequency

```python
product_stats = (
    order_items
    .groupby('product_id')
    .agg(
        avg_price=('price', 'mean'),
        purchase_count=('order_id', 'count')
    )
    .reset_index()
)

product_stats
```

	product_id	avg_price	purchase_count
0	00066f42aeeb9f3007548bb9d3f33c38	101.65	1
1	00088930e925c41fd95ebfe695fd2655	129.90	1
2	0009406fd7479715e4bef61dd91f2462	229.00	1
3	000b8f95fcb9e0096488278317764d19	58.90	2
4	000d9be29b5207b54e86aa1b1ac54872	199.00	1
...	...	...	...
32946	fff6177642830a9a94a0f2cba5e476d1	114.99	2
32947	fff81cc3158d2725c0655ab9ba0f712c	90.00	1
32948	fff9553ac224cec9d15d49f5a263411f	32.00	1
32949	fffdb2d0ec8d6a61f0a0a0db3f25b441	33.99	5
32950	fffe9eeff12fcbd74a2f2b007dde0c58	249.99	1

32951 rows × 3 columns



5) Calculate total revenue generated by each seller and rank them

- SQL Code:

```sql
#5. Calculate total revenue generated by each seller and rank them
SELECT
    s.seller_id,
    SUM(oi.price) AS total_revenue,
    RANK() OVER (ORDER BY SUM(oi.price) DESC) AS revenue_rank
FROM sellers s
JOIN order_items oi
    ON s.seller_id = oi.seller_id
GROUP BY s.seller_id
ORDER BY revenue_rank;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

seller_id	total_revenue	revenue_rank
4869f7a5dfa277a7dca6462dcf3b52b2	229472.6299999981	1
53243585a1d6dc2643021fd1853d8905	222776.04999999952	2
4a3ca9315b744ce9f8e9374361493884	200472.9199999949	3
fa1c13f2614d7b5c4749cbc52fecda94	194042.02999999846	4
7c67e1448b00f6e969d365cea6b010ab	187923.8899999995	5
7e93a43ef30c4f03f38b393420bc753a	176431.86999999982	6
da8622b14eb17ae2831f4ac5b9dab84a	160236.56999999538	7
7a67c85e85bb2ce8582c35f2203ad736	141745.53000000177	8
1025f0e2d44d7041d6cf58b6550e0bfa	138968.5499999995	9
955fee9216a65b617aa5c0531780ce60	135171.69999999914	10

Result 10

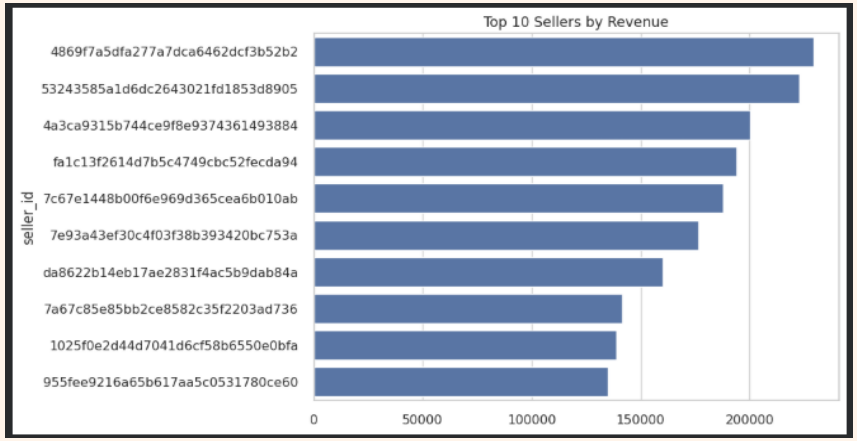
- Python Code:

5) Total revenue per seller + ranking

```python
seller_revenue = order_items.groupby('seller_id')['price'].sum()

seller_rank = seller_revenue.rank(method='min', ascending=False)

result = pd.DataFrame({
    'total_revenue': seller_revenue,
    'revenue_rank': seller_rank
}).sort_values('revenue_rank')

result
```

seller_id	total_revenue	revenue_rank
4869f7a5dfa277a7dca6462dcf3b52b2	229472.63	1.0
53243585a1d6dc2643021fd1853d8905	222776.05	2.0
4a3ca9315b744ce9f8e9374361493884	200472.92	3.0
fa1c13f2614d7b5c4749cbc52fecda94	194042.03	4.0
7c67e1448b00f6e969d365cea6b010ab	187923.89	5.0
...	...	...
34aefe746cd81b7f3b23253ea28bef39	8.00	3091.0
702835e4b785b67a084280efca355756	7.60	3092.0
1fa2d3def6adfa70e58c276bb64fe5bb	6.90	3093.0
77128dec4bec4878c37ab7d6169d6f26	6.50	3094.0
cf6f6bc4df3999b9c6440f124fb2f687	3.50	3095.0

3095 rows × 2 columns



ADVANCE PROBLEMS

1) Moving average of order values for each customer

- SQL Code:

```sql
#1. Moving average of order values for each customer
WITH order_values AS (
    SELECT
        o.order_id,
        o.customer_id,
        o.order_purchase_timestamp,
        SUM(oi.price) AS order_value
    FROM orders o
    JOIN order_items oi
        ON o.order_id = oi.order_id
    GROUP BY o.order_id, o.customer_id, o.order_purchase_timestamp
)
SELECT
    customer_id,
    order_purchase_timestamp,
    order_value,
    AVG(order_value) OVER (
        PARTITION BY customer_id
        ORDER BY order_purchase_timestamp
        ROWS BETWEEN 2 PRECEDING AND CURRENT ROW
    ) AS moving_avg_order_value
FROM order_values;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: | Fetch rows:

customer_id	order_purchase_timestamp	order_value	moving_avg_order_value
00012a2ce6f8dcda20d059ce98491703	2017-11-14 16:08:26	89.8	89.8
000161a058600d5901f007fab4c27140	2017-07-16 9:40:32	54.9	54.9
0001fd6190edaaf884bcaf3d49edf079	2017-02-28 11:06:43	179.99	179.99
0002414f95344307404f0ace7a26f1d5	2017-08-16 13:09:20	149.9	149.9
000379cdec625522490c315e70c7a9fb	2018-04-02 13:42:17	93	93
0004164d20a9e969af783496f3408652	2017-04-12 8:35:12	59.99	59.99
000419c5494106c306a97b5635748086	2018-03-02 17:47:40	34.3	34.3
00046a560d407e99b969756e0b10f282	2017-12-18 11:08:30	120.9	120.9
00050bf6e01e69d5c0fd612f1bcfb69c	2017-09-17 16:04:44	69.99	69.99
000598caf2ef4117407665ac33275130	2018-08-11 12:14:35	1107	1107
0005aefbb696d34b3424dccd0a0e9fd0	2018-06-20 9:46:53	134.9	134.9

Result 15

- Python Code:

1) Moving average of order values per customer

```python
orders['order_purchase_timestamp'] = pd.to_datetime(
    orders['order_purchase_timestamp']
)

order_values = (
    orders
    .merge(order_items, on='order_id', how='inner')
    .groupby(['order_id','customer_id','order_purchase_timestamp'], as_index=False)
    .agg(order_value=('price','sum'))
)

order_values = order_values.sort_values(
    ['customer_id','order_purchase_timestamp']
)

order_values['moving_avg_order_value'] = (
    order_values
    .groupby('customer_id')['order_value']
    .rolling(window=3, min_periods=1)
    .mean()
    .reset_index(level=0, drop=True)
)
```

	order_id	customer_id	order_purchase_timestamp	order_value	moving_avg_order_value
38880	5f79b5b0931d63f1a42989eb65b9da6e	00012a2ce6f8dcda20d059ce98491703	2017-11-14 16:08:26	89.80	89.80
83006	a44895d095d7e0702b6a162fa2dbecad	000161a058600d5901f007fab4c27140	2017-07-16 09:40:32	54.90	54.90
13934	316a104623542e4d75189bb372bc5f8d	0001fd6190edaaf884bcaf3d49edf079	2017-02-28 11:06:43	179.99	179.99
33938	5825ce2e88d53464386f6b0bba99e5ee	0002414f95344307404f0ace7a26f1d5	2017-08-16 13:09:20	149.90	149.90
4187	0ab7fb08086d4af9141453c91878ad7a	000379cdec625522490c315e70c7a9fb	2018-04-02 13:42:17	93.00	93.00
...	...	...	...	...	...
37708	620ddc9fbe3e445676a8e71d4a830821	ffffcb937e9dd47a13f05ecb829014d3a	2018-03-17 00:55:27	78.00	78.00
48740	814d6a3a7c0b32b2ad929ac6328124e9	ffffece9f79fd8c764f843e9951b11341	2018-03-29 16:59:26	54.90	54.90
53932	8c855550908247a7aff50281b92167a8	ffffda5b6d849fbd39689bb92087f431	2018-05-22 13:36:02	47.90	47.90
60026	83b5fc912b2862c5046555dad1483ae9	ffff42319e9b2d713724aa527742af25	2018-06-13 16:57:05	199.90	199.90
80668	d0e7be325a1c986babc4a1cdb91edc03	ffffa3172527765de70084a7e53aaa8	2017-09-02 11:53:32	21.80	21.80

98666 rows × 5 columns

2) Cumulative sales per month for each year

- SQL Code:

```sql
#2. Cumulative sales per month for each year
WITH monthly_sales AS (
    SELECT
        YEAR(o.order_purchase_timestamp) AS year,
        MONTH(o.order_purchase_timestamp) AS month,
        SUM(oi.price) AS monthly_sales
    FROM orders o
    JOIN order_items oi
        ON o.order_id = oi.order_id
    GROUP BY year, month
)
SELECT
    year,
    month,
    monthly_sales,
    SUM(monthly_sales) OVER (
        PARTITION BY year
        ORDER BY month
    ) AS cumulative_sales
FROM monthly_sales
ORDER BY year, month;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

year	month	monthly_sales	cumulative_sales
2016	9	267.36	267.36
2016	10	49507.66000000018	49775.02000000018
2016	12	10.9	49785.92000000018
2017	1	120312.86999999972	120312.86999999972
2017	2	247303.0199999959	367615.8899999956
2017	3	374344.3000000005	741960.1899999961
2017	4	359927.2299999999	1101887.419999996
2017	5	506071.1400000094	1607958.5600000054
2017	6	433038.60000000405	2040997.1600000095
2017	7	498031.48000001034	2539028.6400000197
2017	8	573971.6800000152	3113000.3200000348

Result 14

- Python Code:

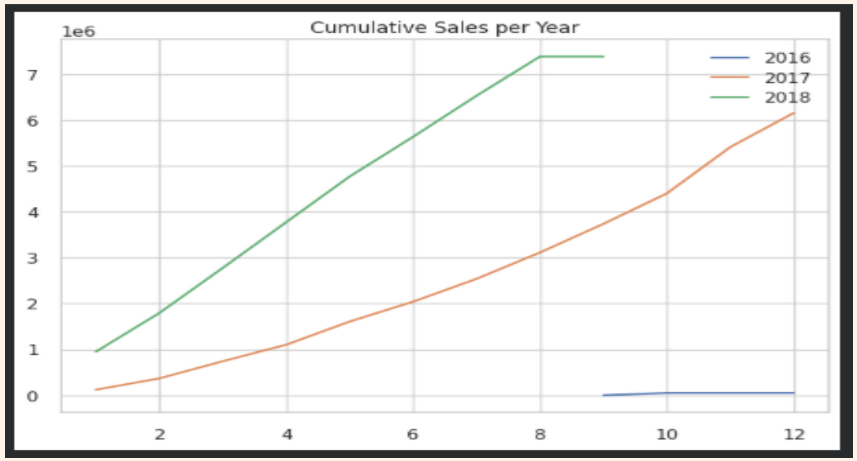
2) Cumulative monthly sales per year

```python
orders['order_purchase_timestamp'] = pd.to_datetime(
    orders['order_purchase_timestamp']
)

sales_data = orders.merge(order_items, on='order_id', how='inner')

sales_data['year'] = sales_data['order_purchase_timestamp'].dt.year
sales_data['month'] = sales_data['order_purchase_timestamp'].dt.month

monthly_sales = (
    sales_data
    .groupby(['year','month'], as_index=False)
    .agg(monthly_sales=('price','sum'))
)

monthly_sales = monthly_sales.sort_values(['year','month'])

monthly_sales['cumulative_sales'] = (
    monthly_sales
    .groupby('year')['monthly_sales']
    .cumsum()
)

monthly_sales
```

	year	month	monthly_sales	cumulative_sales
0	2016	9	267.36	267.36
1	2016	10	49507.66	49775.02
2	2016	12	10.90	49785.92
3	2017	1	120312.87	120312.87
4	2017	2	247303.02	367615.89
5	2017	3	374344.30	741960.19
6	2017	4	359927.23	1101887.42
7	2017	5	506071.14	1607958.56
8	2017	6	433038.60	2040997.16
9	2017	7	498031.48	2539028.64
10	2017	8	573971.68	3113000.32

```python
len(monthly_sales)
```

```
24
```



3) Year-over-Year (YoY) growth rate of total sales

- SQL Code:

```sql
#3. Year-over-Year (YoY) growth rate of total sales
WITH yearly_sales AS (
    SELECT
        YEAR(o.order_purchase_timestamp) AS year,
        SUM(oi.price) AS total_sales
    FROM orders o
    JOIN order_items oi
        ON o.order_id = oi.order_id
    GROUP BY year
)
SELECT
    year,
    total_sales,
    ROUND(
        (total_sales - LAG(total_sales) OVER (ORDER BY year))
        * 100.0 /
        LAG(total_sales) OVER (ORDER BY year),
        2
    ) AS yoy_growth_percentage
FROM yearly_sales;
```

Result Grid | Filter Rows: | Export:

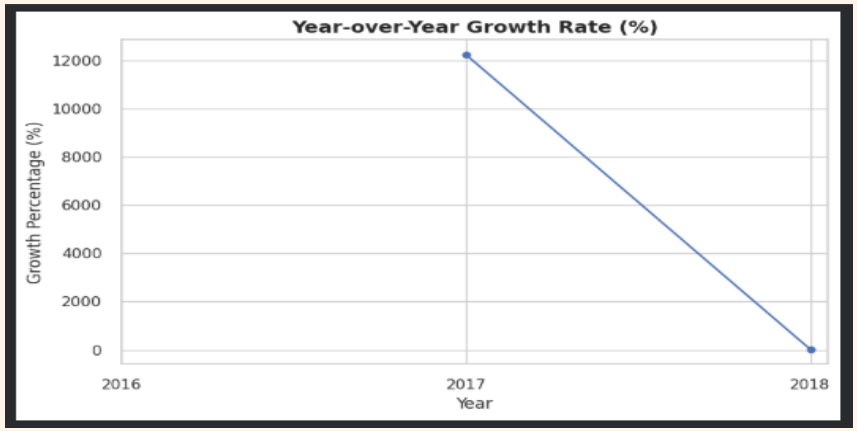
year	total_sales	yoy_growth_percentage
2016	49785.92000000019	NULL
2017	6155806.9800029425	12264.55
2018	7386050.800004353	19.99

- Python Code:

3) Year-over-Year Growth Rate

```python
orders['order_purchase_timestamp'] = pd.to_datetime(
    orders['order_purchase_timestamp']
)

sales_data = orders.merge(order_items, on='order_id', how='inner')

sales_data['year'] = sales_data['order_purchase_timestamp'].dt.year

yearly_sales = (
    sales_data
    .groupby('year', as_index=False)
    .agg(total_sales=('price','sum'))
)

yearly_sales = yearly_sales.sort_values('year')

yearly_sales['previous_year'] = yearly_sales['total_sales'].shift(1)

yearly_sales['yoy_growth_percentage'] = (
    (yearly_sales['total_sales'] - yearly_sales['previous_year'])
    * 100.0
    / yearly_sales['previous_year']
).round(2)

yearly_sales
```

	year	total_sales	previous_year	yoy_growth_percentage
0	2016	49785.92	NaN	NaN
1	2017	6155806.98	49785.92	12264.55
2	2018	7386050.80	6155806.98	19.99



4) Customer retention rate

- SQL Code:

```sql
#4. Customer retention rate
WITH first_orders AS (
    SELECT
        c.customer_unique_id,
        MIN(o.order_purchase_timestamp) AS first_order_date
    FROM orders o
    JOIN customers c
        ON o.customer_id = c.customer_id
    GROUP BY c.customer_unique_id
),
repeat_customers AS (
    SELECT DISTINCT
        fo.customer_unique_id
    FROM orders o
    JOIN customers c
        ON o.customer_id = c.customer_id
    JOIN first_orders fo
        ON c.customer_unique_id = fo.customer_unique_id
    WHERE o.order_purchase_timestamp > fo.first_order_date
      AND o.order_purchase_timestamp <= DATE_ADD(fo.first_order_date, INTERVAL 6 MONTH)
)
SELECT
    ROUND(
        COUNT(DISTINCT rc.customer_unique_id) * 100.0 /
        COUNT(DISTINCT fo.customer_unique_id),
        2
    ) AS retention_rate_percentage
FROM first_orders fo
LEFT JOIN repeat_customers rc
    ON fo.customer_unique_id = rc.customer_unique_id;
```

Result Grid | Filter Rows:

	retention_rate_percentage
▶	2.32

- Python Code:

4) 6-Month Customer Retention Rate

```python
orders['order_purchase_timestamp'] = pd.to_datetime(
    orders['order_purchase_timestamp']
)

orders_customers = orders.merge(customers, on='customer_id', how='inner')

first_orders = (
    orders_customers
    .groupby('customer_unique_id', as_index=False)
    .agg(first_order_date=('order_purchase_timestamp','min'))
)

merged_ret = orders_customers.merge(
    first_orders,
    on='customer_unique_id',
    how='inner'
)

repeat_customers = merged_ret[
    (merged_ret['order_purchase_timestamp'] > merged_ret['first_order_date']) &
    (merged_ret['order_purchase_timestamp'] <=
     merged_ret['first_order_date'] + pd.DateOffset(months=6))
]

retention_rate = round(
    repeat_customers['customer_unique_id'].nunique()
    * 100.0
    / first_orders['customer_unique_id'].nunique(),
    2
)

retention_rate
```

```
2.32
```

5) Top 3 customers by spending in each year

- SQL Code:

```sql
#5. Top 3 customers by spending in each year
WITH yearly_customer_spend AS (
    SELECT
        YEAR(o.order_purchase_timestamp) AS year,
        c.customer_unique_id,
        SUM(oi.price) AS total_spent
    FROM orders o
    JOIN customers c
        ON o.customer_id = c.customer_id
    JOIN order_items oi
        ON o.order_id = oi.order_id
    GROUP BY year, c.customer_unique_id
),
ranked_customers AS (
    SELECT *,
            RANK() OVER (
                PARTITION BY year
                ORDER BY total_spent DESC
            ) AS rank_in_year
    FROM yearly_customer_spend
)
SELECT
    year,
    customer_unique_id,
    total_spent
FROM ranked_customers
WHERE rank_in_year <= 3
ORDER BY year, rank_in_year;
```

Result Grid | Filter Rows: | Export: | Wrap

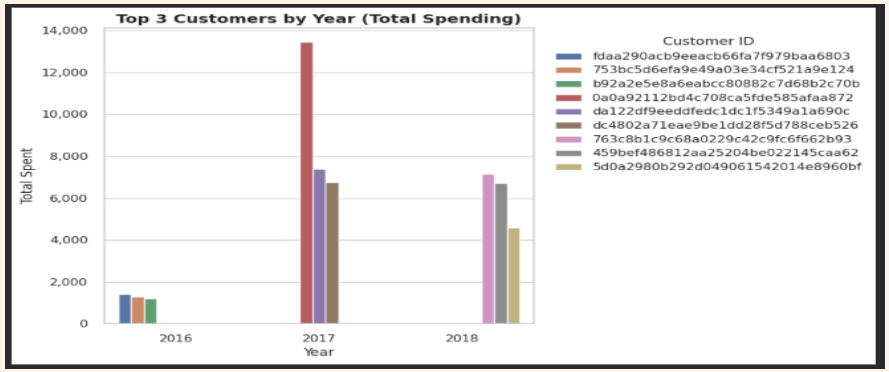
year	customer_unique_id	total_spent
2016	fdaa290acb9eeacb66fa7f979baa6803	1399
2016	753bc5d6efa9e49a03e34cf521a9e124	1299.99
2016	b92a2e5e8a6eabcc80882c7d68b2c70b	1199
2017	0a0a92112bd4c708ca5fde585afaa872	13440
2017	da122df9eeddfedc1dc1f5349a1a690c	7388
2017	dc4802a71eae9be1dd28f5d788ceb526	6735
2018	763c8b1c9c68a0229c42c9fc6f662b93	7160
2018	459bef486812aa25204be022145caa62	6729
2018	5d0a2980b292d049061542014e8960bf	4599.9

- Python Code:

5) Top 3 customers by spending each year

```python
merged_sql_logic = (
    orders
    .merge(customers[['customer_id', 'customer_unique_id']],
           on='customer_id', how='inner')
    .merge(order_items[['order_id', 'price']],
           on='order_id', how='inner')
)

merged_sql_logic['year'] = merged_sql_logic['order_purchase_timestamp'].dt.year

yearly_customer_spend = (
    merged_sql_logic
    .groupby(['year', 'customer_unique_id'])['price']
    .sum()
    .reset_index(name='total_spent')
)

yearly_customer_spend['rank_in_year'] = (
    yearly_customer_spend
    .groupby('year')['total_spent']
    .rank(method='dense', ascending=False)
)

top_3_customers = yearly_customer_spend[
    yearly_customer_spend['rank_in_year'] <= 3
].sort_values(['year', 'rank_in_year'])

top_3_customers
```

	year	customer_unique_id	total_spent	rank_in_year
303	2016	fdaa290acb9eeacb66fa7f979baa6803	1399.00	1.0
138	2016	753bc5d6efa9e49a03e34cf521a9e124	1299.99	2.0
221	2016	b92a2e5e8a6eabcc80882c7d68b2c70b	1199.00	3.0
2042	2017	0a0a92112bd4c708ca5fde585afaa872	13440.00	1.0
37231	2017	da122df9eeddfedc1dc1f5349a1a690c	7388.00	2.0
37801	2017	dc4802a71eae9be1dd28f5d788ceb526	6735.00	3.0
67370	2018	763c8b1c9c68a0229c42c9fc6f662b93	7160.00	1.0
67352	2018	459bef486812aa25204be022145caa62	6729.00	2.0
62888	2018	5d0a2980b292d049061542014e8960bf	4599.90	3.0



Conclusion

- This project demonstrates full-stack analytical capability by combining SQL, Python, window functions, and validation methodology to transform raw transactional data into strategic business insights.
- Consistent cross-validation between SQL and Python ensures analytical accuracy and production-level reliability.